🐾 Pet-Care Connecting System

Viva Preparation — Part 2: API-to-ML Integration & Technical Deep Dive

Q1 What is the "API to ML" part? Why is it important?

The main special thing about our project is we use **external AI/ML APIs** to do health analysis on pets. We don't train our own model — we call existing powerful ML models through their REST APIs.

How it works: User uploads a **pet photo + symptoms** → our backend sends it to **Google Gemini API** (a multimodal AI) → Gemini analyzes the image and text → returns a preliminary diagnosis → we save it → a real vet can review it later.

So "API to ML" means: **our backend acts as a middleman between the user and the AI model**. The frontend never talks to Gemini directly.

Q2 Why API calls instead of building your own ML model?

Building our own model would need huge labeled datasets, GPU resources, and ML expertise. Instead, we use **pre-trained models via API** (Google Gemini, Groq LLaMA) that are already trained on billions of data points.

This is actually how most real-world apps work today — they consume ML-as-a-Service from Google, OpenAI, etc.

Q3 Walk through the full consultation flow — step by step.

1. **User (Frontend):** Selects pet, uploads photo, types symptoms, clicks "Start Analysis"
2. **API Call:** Frontend sends `POST /api/consultations/analyze` with petId, base64Image, mimeType, symptoms + JWT header
3. **Controller:** `ConsultationController.analyze()` receives request, extracts authenticated user
4. **Service:** Validates pet ownership, validates symptoms not empty, builds AI prompt, calls `PetHealthMLService.analyzeWithImage()`
5. **ML Service → Gemini API:** Sends HTTP POST to `generativelanguage.googleapis.com` with text prompt + base64 image as `inlineData`

6. **Response:** Extracts text from `response → candidates[0] → content → parts[0] → text` . If Gemini fails → fallback to Groq API
7. **Save:** Creates `Consultation` entity with `aiPreliminaryDiagnosis` = AI response, `status = AI_REVIEWED` , saves to MySQL
8. **Return:** JSON response sent back → frontend shows the diagnosis + adds to history
9. **Vet Review (later):** Vet logs in → sees AI-reviewed cases → adds final diagnosis + treatment → status becomes `VET_REVIEWED`

Q4 What is Google Gemini API?

Gemini is Google's multimodal AI model — it can understand text, images, audio, and video. We use **Gemini 2.0 Flash** which is fast, free-tier available, and supports vision (image analysis).

Endpoint: `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent`

We authenticate using an API key as a URL parameter: `?key=OUR_API_KEY`

Q5 What is Groq API? Why the fallback?

Groq hosts open-source models like **LLaMA 3.3 70B** (by Meta). Known for super fast inference.

We use it as a **fallback** — if Gemini API fails (rate limit, downtime, error), the system automatically switches to Groq:

```
User request → Try Gemini →
    Success → return result
    Fail → Try Groq →
        Success → return result
        Fail → "Service unavailable" message
```

Limitation: Groq fallback only uses text (not images) — that's why Gemini is primary.

Endpoint: `https://api.groq.com/openai/v1/chat/completions` (OpenAI-compatible format, Bearer token auth)

Q6 Show the actual ML API calling code.

Key method from `PetHealthMLService.java` :

```
public String analyzeWithImage(String base64Image, String mimeType, String prompt) {
    // Skip to fallback if no API key
```

```
    if (mlApiKey == null || mlApiKey.isBlank())
        return analyzeFallback(prompt, prompt);

    // Build Gemini request body
    Map requestBody = new HashMap();
    requestBody.put("contents", List.of(
        Map.of("parts", List.of(
            Map.of("text", briefPrompt),          // text prompt
            Map.of("inlineData", Map.of(           // the image
                "mimeType", mimeType,
                "data", base64Image
            ))
        ))
    ));

    // Call Gemini using RestTemplate
    ResponseEntity response = restTemplate.exchange(
        ML_API_URL + "?key=" + mlApiKey,
        HttpMethod.POST, entity, ...);

    // Extract text, fallback if empty
    String content = extractMLContent(response.getBody());
    return content != null ? content : analyzeFallback(prompt, prompt);
}
```

`RestTemplate` = Spring's HTTP client for making REST calls to external APIs.

Q7 Chatbot vs Consultation — what's the difference?

Feature	Chatbot	Consultation
Input	Text message only	Image + symptoms + pet selection
AI Model	Groq LLaMA 3.3	Google Gemini 2.0 Flash (primary)
Saved to DB?	No — conversational only	Yes — full record in consultations table
Vet review?	No	Yes — vets add final diagnosis
Use case	Quick questions like "is chocolate bad for dogs?"	Serious health concerns with photo

---

Q8 What is RestTemplate?

`RestTemplate` is a **synchronous HTTP client** from Spring Framework. We use it to call external APIs (Gemini, Groq) from our Java backend. It's built into Spring — no extra dependencies needed.

Q9 What is the consultation status flow?

```
PENDING → AI_REVIEWED → VET_REVIEWED → RESOLVED
```

- **PENDING** — just created, waiting for AI
- **AI_REVIEWED** — AI gave preliminary diagnosis, waiting for vet
- **VET_REVIEWED** — vet added final diagnosis + treatment
- **RESOLVED** — case closed

Q10 How do you handle errors when AI API fails?

Level	What happens
ML Service	Try-catch for `HttpClientErrorException` (rate limit, 400/500) and general `Exception` (network/timeout)
Fallback	If Gemini fails → try Groq. If both fail → return friendly message: "Unable to analyze"
Frontend	Catches error responses, shows alert instead of crashing
Global Handler	`GlobalExceptionHandler.java` catches unhandled exceptions, returns proper HTTP codes

---

Q11 How do you secure the API keys?

Keys stored in `application.properties`. Injected with `@Value("${google.gemini.api.key:}")` — the `:` means default to empty if missing.

For production: use environment variables, Spring Cloud Config, or AWS Secrets Manager. Never commit keys to GitHub.

> ⚠️ The keys are never exposed to the frontend — all AI calls go through our backend.

Q12 What HTTP methods and status codes does your API use?

Method	Usage	Example
`GET`	Fetch data	`GET /api/pets`

`POST`	Create / action	`POST /api/consultations/analyze`
`PUT`	Update	`PUT /api/consultations/3/vet-review`
`DELETE`	Remove	`DELETE /api/consultations/3`

Code	Meaning	When
200	OK	Successful request
201	Created	New resource created
204	No Content	Successful delete
400	Bad Request	Missing fields
401	Unauthorized	Invalid/expired JWT
403	Forbidden	Wrong role
404	Not Found	Resource doesn't exist

Q13 What are Java Records?

Records (Java 16+) are compact immutable data classes. Auto-generates constructor, getters, equals, hashCode, toString.

```
public record DiagnosisRequest(Long petId, String base64Image,
                               String mimeType, String symptoms) {}
```

We used them for request DTOs in controllers — much cleaner than full classes.

Q14 How does image upload work?

1. Frontend reads image with `FileReader.readAsDataURL(file)` → gets base64 string
2. Splits to extract just the base64 part: `imagePreview.split(',')[1]`
3. Sends base64 string + mimeType in JSON body to backend
4. Backend forwards it directly to Gemini API as `inlineData`

Why base64? Simpler than multipart form handling, and Gemini accepts inline base64 directly.

Q15 Explain the Spring Security filter chain.

```
HTTP Request
  → CORS Filter (allows localhost:3000)
  → CSRF Filter (disabled — we use JWT)
  → JwtAuthFilter (our custom filter)
      → Extract token from Authorization header
      → Validate token (signature + expiry)
      → Load user from DB
      → Set authentication in SecurityContext
  → Authorization Check
      → /api/auth/** → permitAll (no auth needed)
      → everything else → must be authenticated
  → Controller method executes
```

Our `JwtAuthFilter` extends `OncePerRequestFilter` and sits before `UsernamePasswordAuthenticationFilter`.

Q16 "Is this really ML?" — how to answer

"We use ML through **API integration**. The ML part is the Google Gemini model — trained on massive datasets with vision capabilities. What we built is the **integration layer** — the backend that takes user input, constructs the prompt, sends it to the model's API, processes the response, stores results, handles errors with fallback, and manages the AI-review → vet-review workflow. This is called **ML-as-a-Service consumption**."

Q17 Quick-fire tricky questions

Question	Short Answer
Why Spring Boot, not Node.js?	Java is strongly typed — better for large apps. Spring has built-in security, JPA, validation.
Why MySQL, not MongoDB?	Our data is relational (pets belong to users, appointments link pets to vets). SQL handles FK and joins better.
Both AI APIs fail?	Returns friendly message: "Unable to analyze. Please describe the issue to your vet." App doesn't crash.
Is AI diagnosis accurate?	It's preliminary only — app clearly says "Always consult a licensed vet." Real value = vet review after.
Why TypeScript?	Type safety at compile time, better autocomplete, cleaner code with interfaces.

How prevent SQL injection?	JPA uses parameterized queries by default. We never write raw SQL.
Entity vs DTO?	Entity maps to a DB table (`@Entity`). DTO is just a data carrier between layers — no JPA annotations.